Mobile Robot Path Planning in Cluttered Environments Using Cuckoo Search with Lévy Flights

Author Name [1], Author Name [2] [1]Department of Electronics and Communication Engineering, Institution Name, City, Country [2]Department of Computer Science, Institution Name, City, Country Email: author@institution.edu

Abstract

Path planning is a core capability of autonomous mobile robots, requiring a collision-free route from a start to a goal configuration that minimizes length, energy, or travel time. Classical grid-search and potential-field methods struggle with scalability and local minima in cluttered environments. This paper presents a path planner based on Cuckoo Search (CS), a nature-inspired metaheuristic that combines Lévy-flight exploration with a nest-abandonment mechanism. Candidate paths are encoded as sequences of intermediate waypoints and evaluated by a penalty-based fitness function that jointly accounts for path length, obstacle collision, and smoothness. A cubic-spline post-processing step produces smooth trajectories. The planner is evaluated in simulation against A*, Particle Swarm Optimization (PSO), and Genetic Algorithm (GA) across several obstacle densities. The experimental protocol and result tables are provided so that results can be reproduced and reported.

Index Terms—Cuckoo search, Lévy flight, metaheuristic optimization, mobile robot, path planning, obstacle avoidance.

I. Introduction

Autonomous mobile robots are deployed in warehouses, hospitals, agriculture, and search-and-rescue, where they must navigate among static and dynamic obstacles. The path planning problem asks for a feasible path between two points that optimizes one or more criteria while avoiding collisions [1].

Existing approaches fall into several families. Graph-search methods such as Dijkstra's algorithm [2] and A* [3] are complete and optimal on a given discretization, but their cost grows rapidly with map resolution and dimensionality. Artificial potential fields [4] are computationally cheap but suffer from local minima. Sampling-based planners such as

RRT [5] scale well but yield jagged, non-optimal paths that need post-processing. Metaheuristics, including Genetic Algorithms, Particle Swarm Optimization [6], and Cuckoo Search [7], treat path planning as a continuous optimization problem and need no explicit map discretization.

Cuckoo Search, proposed by Yang and Deb [7], is inspired by the brood parasitism of cuckoo species and uses Lévy flights for global exploration. It has few control parameters (population size and discovery probability), which makes it attractive for tuning-light deployment. This paper makes the following contributions:

1. A waypoint-based encoding of robot paths suitable for Cuckoo Search.
2. A multi-term fitness function combining length, collision penalty, and smoothness.
3. A complete planner including repair and spline smoothing steps.
4. A reproducible benchmarking protocol against A*, PSO, and GA.

The remainder of the paper is organized as follows. Section II reviews related work, Section III formulates the problem, Section IV describes the proposed method, Section V gives the experimental setup, Section VI presents results and discussion, and Section VII concludes.

II. Related Work

Graph-based planners remain the baseline in structured environments. A* uses a heuristic to guide search [3], and its variants (D*, Theta*) address replanning and any-angle motion. Potential-field methods [4] model the goal as an attractor and obstacles as repulsors, but are prone to oscillation and trapping.

Evolutionary and swarm methods have been widely applied to path planning. GA-based planners use crossover and mutation over waypoint chromosomes; PSO-based planners update waypoint positions using personal and global best experience [6]. These methods are flexible regarding cost functions but can converge prematurely.

Cuckoo Search was introduced for continuous optimization in [7] and applied to engineering design problems in [8], where it was reported to compare favorably with PSO and GA on several benchmarks. Its use of Lévy flights, whose heavy-tailed step distribution mixes many small local moves with occasional long jumps, supports a good balance between exploitation and exploration. Lévy-distributed steps are commonly generated using Mantegna's algorithm [9]. This paper adapts CS to the constrained geometry of robot navigation.

III. Problem Formulation

Let the workspace be $W \subset \mathbb{R}^2$, with a set of obstacles $O = \{O_1, \ldots, O_m\}$, modeled as circles or polygons. The free space is $W_{free} = W \setminus \cup O_i$. Given a start point **s** and goal point **g**, a path is an ordered sequence $P = (\mathbf{p}_0, \mathbf{p}_1, \ldots, \mathbf{p}_{n+1})$ with $\mathbf{p}_0 = \mathbf{s}$ and $\mathbf{p}_{n+1} = \mathbf{g}$, where $\mathbf{p}_1 \ldots \mathbf{p}_n$ are the n intermediate waypoints, and consecutive points are joined by straight segments.

The objective is to find the path that minimizes

$$F(P) = w_1 \cdot L(P) + w_2 \cdot C(P) + w_3 \cdot S(P) \quad (1)$$

where:

- **L(P)** is the Euclidean length: $L(P) = \sum_{i=0}^{n} \|\mathbf{p}_{i+1} - \mathbf{p}_i\|$ (2)
- **C(P)** is the collision penalty: $C(P) = \sum_{i=0}^{n} \sum_{j=1}^{m} \max(0, r_j + d_{safe} - \text{dist}(seg_i, O_j))$ (3), where r_j is the obstacle radius, d_{safe} is a safety margin, and $\text{dist}(seg_i, O_j)$ is the distance from segment i to the center of obstacle j.
- **S(P)** is the smoothness term: $S(P) = \sum_{i=1}^{n} |\theta_i|$ (4), where θ_i is the turning angle at waypoint $\mathbf{p}_i$.

The weights satisfy $w_2 \gg w_1$ so that any path with collisions is strongly dominated by collision-free paths. The robot is modeled as a point with a safety margin d_{safe} to account for its footprint.

IV. Proposed Method

A. Cuckoo Search Principles

Cuckoo Search follows three idealized rules [7]:

1. Each cuckoo lays one egg at a time and places it in a randomly chosen nest.
2. The best nests (highest-quality solutions) carry over to the next generation.
3. The number of host nests N is fixed, and a host discovers an alien egg with probability $p_a \in [0, 1]$; discovered nests are abandoned and replaced.

In the optimization setting, each nest is a candidate solution and each egg's quality is the inverse of its fitness value.

B. Solution Encoding

A nest encodes a path with n intermediate waypoints as a vector $X = [x_1, y_1, x_2, y_2, \ldots, x_n, y_n]$, with each coordinate bounded by the workspace limits. To exploit the geometry of the problem, the initial population is generated by perturbing points along the straight line from **s** to **g** with Gaussian noise, which gives a population concentrated near plausible routes while still diverse. Using waypoints ordered by their projection onto the start–goal line reduces self-intersecting paths.

C. Lévy Flight Update

A new solution for nest i at generation t is produced by a Lévy flight:

$$X_i^{(t+1)} = X_i^{(t)} + \alpha \oplus \text{Lévy}(\lambda) \quad (5)$$

where $\alpha > 0$ is the step-size scale and $\oplus$ denotes entry-wise multiplication. Lévy steps are drawn using Mantegna's algorithm [9]:

$$s = u / |v|^{1/\beta}, \; u \sim N(0, \sigma_u^2), \; v \sim N(0, 1) \quad (6)$$

$$\sigma_u = \{ \Gamma(1+\beta) \sin(\pi\beta/2) / (\Gamma((1+\beta)/2) \cdot \beta \cdot 2^{(\beta-1)/2}) \}^{1/\beta} \quad (7)$$

with $\beta = 1.5$ commonly used. To bias moves toward good regions, the step is scaled by the distance to the current best nest:

$$\text{step} = \alpha \cdot s \odot (X_i - X_{best}) \quad (8)$$

D. Nest Abandonment

After the Lévy-flight stage, a fraction p_a of the worst nests is replaced by new solutions generated by a biased random walk:

$$X_i^{(t+1)} = X_i^{(t)} + r \cdot H(p_a - \varepsilon) \odot (X_j^{(t)} - X_k^{(t)}) \quad (9)$$

where X_j and X_k are two randomly permuted nests, r and ε are uniform random numbers in [0, 1], and $H(\cdot)$ is the Heaviside function. Greedy selection keeps a new solution only if it improves fitness.

E. Repair and Smoothing

Candidates are clipped to workspace bounds. After convergence, the best path is smoothed with a cubic spline through its waypoints, and the smoothed path is re-checked for collisions; if a collision is found, the unsmoothed polyline is returned.

F. Algorithm

Algorithm 1: Cuckoo Search Path Planner

1. Input: map, **s**, **g**, N, n, p_a, α, β, T_max.
2. Initialize N nests along the s–g line with Gaussian perturbation.
3. Evaluate F for all nests; identify X_best.
4. For t = 1 to T_max:
   - a. For each nest, generate a new solution by Lévy flight (Eq. 5–8); apply greedy replacement.
   - b. Abandon a fraction p_a of the worst nests and regenerate them (Eq. 9); apply greedy replacement.
   - c. Update X_best.
5. Smooth X_best with a cubic spline and validate.
6. Output the final path.

The per-iteration complexity is $O(N \cdot n \cdot m)$ for N nests, n waypoints, and m obstacles, because every segment is checked against every obstacle.

V. Experimental Setup

A. Environments

Simulations are performed on 2-D maps of 100 × 100 units with circular obstacles. Three scenarios are used: **Scenario 1** (sparse, 5 obstacles), **Scenario 2** (moderate, 12 obstacles), and **Scenario 3** (dense, 25 obstacles, including a narrow passage). Start and goal are fixed at (5, 5) and (95, 95).

B. Compared Methods and Parameters

The proposed CS planner is compared with A* (8-connected grid, resolution 1 unit), PSO, and GA. Table I gives the suggested parameters; population size and iteration counts are equalized to give a comparable number of fitness evaluations.

TABLE I. Algorithm Parameters

Parameter	CS	PSO	GA
Population size	40	40	40

Parameter	CS	PSO	GA
Max iterations	200	200	200
Waypoints (n)	6	6	6
Method-specific	p_a = 0.25, β = 1.5, α = 0.01	w = 0.7, $c_1 = c_2 = 1.5$	p_c = 0.8, p_m = 0.05

Fitness weights are $w_1 = 1$, $w_2 = 1000$, $w_3 = 0.5$, and d_safe = 1.0. Each stochastic method is run 30 independent times per scenario.

C. Performance Metrics

Metrics are: (i) mean and best path length, (ii) standard deviation of path length across runs, (iii) success rate (fraction of runs yielding collision-free paths), (iv) mean number of turning angle degrees (smoothness), and (v) computation time in seconds.

VI. Results and Discussion

> **Note:** The values below are placeholders. Run the planner implementation with the settings in Section V and replace each bracketed entry with measured values before submission. No results are claimed here.

TABLE II. Path Length (mean ± std over 30 runs)

Scenario	A*	GA	PSO	CS (proposed)
1 (sparse)	[ ]	[ ]	[ ]	[ ]
2 (moderate)	[ ]	[ ]	[ ]	[ ]
3 (dense)	[ ]	[ ]	[ ]	[ ]

TABLE III. Success Rate (%) and Computation Time (s)

Scenario	Metric	A*	GA	PSO	CS (proposed)
1	Success / Time	[ ]	[ ]	[ ]	[ ]
2	Success / Time	[ ]	[ ]	[ ]	[ ]
3	Success / Time	[ ]	[ ]	[ ]	[ ]

Suggested figures: (Fig. 1) the three test maps with the paths found by each method; (Fig. 2) convergence curves of best fitness versus iteration, averaged over runs; (Fig. 3) box plots of path length across runs.

Discussion Guide

When reporting results, the following points should be analyzed:

- **Convergence behavior:** Lévy flights are expected to help escape local optima in dense maps, which should appear as a continued fitness decline after PSO stagnates. This should be verified, not assumed.
- **Parameter sensitivity:** Vary $p_a \in \{0.1, 0.25, 0.5\}$ and α to show the effect on convergence and success rate.
- **Optimality gap:** A* gives the optimal grid path; report the percentage gap of the metaheuristics against it.
- **Runtime:** Metaheuristics typically cost more than A* in small maps but their cost grows with the number of waypoints rather than the grid size; report timing honestly.
- **Statistical testing:** Use a non-parametric test (e.g., Wilcoxon rank-sum) to assess the significance of differences across the 30 runs.

Limitations

The planner as formulated handles static obstacles in 2-D with a point-robot model. Kinematic and dynamic constraints (non-holonomic motion, velocity limits) are not enforced, and results from simulation may not transfer directly to hardware.

VII. Conclusion and Future Work

This paper presented a Cuckoo Search path planner for mobile robots using a waypoint encoding, a penalty-based fitness function, Lévy-flight search, and spline smoothing, together with a reproducible evaluation protocol. Future work includes extension to dynamic obstacles with online replanning, 3-D environments for aerial robots, multi-objective formulations (length, energy, risk) using multi-objective CS, hybridization with local search, and validation on a physical platform using ROS.

Appendix: Minimal Python Implementation Sketch

```
import numpy as np
from math import gamma, sin, pi

def levy(dim, beta=1.5):
    sigma = (gamma(1+beta)*sin(pi*beta/2) /
             (gamma((1+beta)/2)*beta*2**((beta-1)/2)))**(1/beta)
    u = np.random.normal(0, sigma, dim)
    v = np.random.normal(0, 1, dim)
    return u / np.abs(v)**(1/beta)

def cuckoo_search(fitness, lb, ub, dim, N=40, T=200, pa=0.25, alpha=0.01):
    nests = np.random.uniform(lb, ub, (N, dim))
    fit = np.array([fitness(x) for x in nests])
    best = nests[fit.argmin()].copy()
    for _ in range(T):
        for i in range(N):
            new = np.clip(nests[i] + alpha*levy(dim)*(nests[i]-best), lb, ub)
            f = fitness(new)
            if f < fit[i]:
                nests[i], fit[i] = new, f
        # abandon worst fraction pa
        k = int(pa*N)
        for i in np.argsort(fit)[-k:]:
            j, m = np.random.randint(N, size=2)
            new = np.clip(nests[i] + np.random.rand()*(nests[j]-nests[m]),
lb, ub)
            f = fitness(new)
            if f < fit[i]:
                nests[i], fit[i] = new, f
        best = nests[fit.argmin()].copy()
    return best, fit.min()
```

References

[1] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2006.

[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[3] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, Jul. 1968.

[4] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Int. J. Robot. Res.*, vol. 5, no. 1, pp. 90–98, 1986.

[5] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Iowa State Univ., Tech. Rep. TR 98-11, 1998.

[6] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE Int. Conf. Neural Networks*, Perth, Australia, 1995, pp. 1942–1948.

[7] X.-S. Yang and S. Deb, "Cuckoo search via Lévy flights," in *Proc. World Congr. Nature & Biologically Inspired Computing (NaBIC)*, Coimbatore, India, 2009, pp. 210–214.

[8] X.-S. Yang and S. Deb, "Engineering optimisation by cuckoo search," *Int. J. Math. Model. Numer. Optim.*, vol. 1, no. 4, pp. 330–343, 2010.

[9] R. N. Mantegna, "Fast, accurate algorithm for numerical simulation of Lévy stable stochastic processes," *Phys. Rev. E*, vol. 49, no. 5, pp. 4677–4683, 1994.